Frontend Recreation Plan

AI-Orchestration Content Factory

Component	Tech Stack	Lines of Code
Frontend (apps/web)	React 19, TypeScript 5.9, Tailwind CSS v4, Vite 8, SWR, DnD Kit	~3,830 (68 files)
API Layer (apps/api)	FastAPI, SSE, Supabase Realtime, httpx	~2,500 (16 files)
Backend Packages	Python, LangGraph, Supabase, FreeRouter v3.1	~35,000+ (Python)
FreeRouter	Standalone LLM proxy (LiteLLM), 2 providers, 7 routes	Independent

Prepared for: Hassan Arif

Repository: github.com/HassanArif-collab/AI-Orchestration

Branch: codebase-audit-finding-fixes

April 2026

Version 3.0 - Comprehensive Implementation Guide

Table of Contents

1. Executive Summary

This document provides a comprehensive implementation guide for rebuilding the frontend of the AI-Orchestration project (Content Factory), targeting the codebase-audit-finding-fixes branch. The analysis covers all 68 frontend source files across the React application (apps/web), identifying 7 critical bugs, 14 high-priority issues, and 15 medium-priority improvements. Based on this thorough audit, the document presents a detailed 10-chunk recreation plan designed to rebuild the frontend from scratch in independently testable, incrementally deliverable units, complete with actual TypeScript code examples, database schema references, SSE event protocols, Supabase Realtime integration patterns, and architectural decision rationale.

The current frontend is a Kanban-based dashboard for managing an AI-powered YouTube content pipeline. It integrates with a FastAPI backend via REST endpoints, SSE streaming for chat, and Supabase Realtime WebSockets for live card updates. While the underlying concept is solid, the AI-generated implementation suffers from pervasive architectural issues: uncontrolled re-rendering in streaming components, fragile focus management in modals, missing error boundaries, inconsistent API contracts, and a deprecated drag-and-drop library. These are not isolated bugs but systemic patterns that resist piecemeal patching, necessitating a full selective recreation of the frontend layer.

The recommended approach is a selective recreation: rebuild apps/web and apps/api from the ground up using modern patterns, while surgically fixing the identified bugs in the packages layer (which contains approximately 35,000 lines of complex, mostly correct business logic written in Python using LangGraph). Each chunk in the plan is designed to be independently verifiable against the existing backend, ensuring continuous validation throughout the rebuild process. The plan includes a complete Zod-based type system matching the backend database schema exactly, a reusable SSE client with automatic reconnection, a Zustand-based global UI state management layer, and a comprehensive error boundary strategy.

Metric	Current State	Target State
Frontend Files	68 files, ~3,830 LOC	Clean rebuild, ~2,500 LOC
Critical Bugs	7 identified	0 at each chunk gate
High-Priority Issues	14 identified	0 at each chunk gate
Recreation Chunks	N/A	10 independent chunks
API Endpoints	14 (inconsistent contracts)	14 (strict TypeScript types)
SSE/WebSocket Streams	2 (fragile parsing)	2 (robust with reconnection)
Dependencies Audit	3 outdated/deprecated	All current (April 2026)

Table 1. Frontend Recreation Key Metrics

2. Current Architecture Analysis

2.1 Technology Stack

The frontend is built on a modern but bleeding-edge stack. React 19.2 and Vite 8 are both very recent releases, and while they provide excellent developer experience and performance characteristics, the ecosystem tooling has not fully caught up. Tailwind CSS v4 uses a CSS-first configuration approach (no tailwind.config.js), which is correct but means that all team members must be familiar with the new paradigm. The application uses SWR for data fetching, @dnd-kit for drag-and-drop on the Kanban board, and react-markdown with remark-gfm for rendering markdown content. Supabase provides both the PostgreSQL database and the Realtime WebSocket layer for live updates.

Technology	Version	Purpose	Risk	Audit
React	19.2.4	UI framework	High	Current
Vite	8.0.1	Build tool	High	Current
TypeScript	5.9.3	Type safety	Low	Current
Tailwind CSS	4.2.2	Styling	Medium	Current
SWR	2.4.1	Data fetching	Low	Current
@dnd-kit/core	6.3.1	Drag-and-drop	High	DEPRECATED
Supabase JS	2.100.1	DB + Realtime	Low	Current
react-markdown	10.1.0	Markdown render	Low	Current

Table 2. Frontend Technology Stack with Audit Status

The most critical finding from the dependency audit is that @dnd-kit/core (^6.3.1) and @dnd-kit/sortable (^8.0.0) have been deprecated and merged into a single unified package @dnd-kit/react (^1.0.0) in late 2025. The old packages will not receive security updates or bug fixes, and the API surface has changed significantly. The new @dnd-kit/react package provides a cleaner API with better TypeScript support, improved accessibility, and built-in keyboard navigation that was missing from the old core package. Continuing to use the old packages creates both a security liability and a migration debt that will only grow over time.

2.2 Component Hierarchy

The application is organized as a single-page layout with no client-side routing. The component tree has a maximum depth of 4 levels, which is reasonable. The layout consists of a Header (topic discovery trigger), a central Kanban Board (6 columns with draggable cards), and a right-side Sidebar with 6 tabs (Chat, YouTube, Quota, Models, Skills, Knowledge Base). A CardDrawer overlays the board when a card is clicked, showing detailed information, agent activity logs, script previews, and the review/approval interface.

The data flow architecture uses two parallel patterns: Supabase Realtime WebSocket subscriptions for live Kanban card updates (managed through SWR), and traditional REST API calls for pipeline operations, quota monitoring, and chat streaming. The SSE streaming for chat uses raw fetch() + ReadableStream rather than the EventSource API, which is the correct approach since EventSource only supports GET requests and the chat endpoint requires POST with a body. Agent thoughts are received through a separate Supabase Realtime subscription on the agent_thoughts table.

Level	Components	Count
0 (Root)	MainLayout	1
1 (Layout)	Header, Board, Sidebar, ToastContainer	4
2 (Sections)	Column, Card, CardDrawer, ChatPanel, YouTubePanel, QuotaPanel, ModelRegistry, SkillViewer, KnowledgeBase	9
3 (Widgets)	StatusBadge, EmptyState, ProgressBar, AgentLog, ChatMessage, ScriptViewer, ReviewPanel, PublishConfirmModal, CompetitorCard, OwnStats	10
4 (Leaf)	ReactMarkdown (in multiple parents)	1

Table 3. Component Hierarchy Depth Map

2.3 Data Flow Architecture

The application employs a dual-channel data architecture that creates both flexibility and complexity. The primary data channel for Kanban cards and agent thoughts flows through Supabase Realtime (WebSocket) with SWR as the caching and revalidation layer. When a card is created, updated, or deleted in the backend, the Supabase postgres_changes trigger fires, the WebSocket delivers the event to the frontend, and SWR automatically revalidates the relevant cache key. This means the UI updates in near-real-time without explicit polling for card operations.

The secondary data channel handles pipeline state, quota monitoring, and chat streaming. Pipeline state for a specific card is polled via SWR every 5 seconds (usePipelineState), while provider quotas are similarly polled every 5 seconds (useQuota). The chat system uses Server-Sent Events (SSE) over a POST request to /api/chat/stream, with the response parsed as a ReadableStream. This dual-channel approach is architecturally sound but introduces synchronization challenges: for example, a card moving from "review" status back to "drafting" after rejection is reflected via Supabase Realtime, but the pipeline state poller may still show stale state for up to 5 seconds.

Data Source	Transport	Hook/Module	Cache Key
Kanban Cards	Supabase Realtime + SWR	useCards	kanban-cards
Agent Thoughts	Supabase Realtime	useAgentStream	N/A (append-only)

Data Source	Transport	Hook/Module	Cache Key
Pipeline State	REST + SWR (5s poll)	usePipelineState	pipeline-state-{id}
Provider Quota	REST + SWR (5s poll)	useQuota	provider-quota
Chat Stream	SSE (POST + ReadableStream)	useChat	N/A (streaming)
YouTube Analytics	REST + SWR (5min poll)	useYouTube	competitor-videos

Table 4. Data Flow Architecture

3. Bug Catalog

The following tables present a comprehensive catalog of all identified bugs and issues in the frontend codebase, organized by severity. Each issue includes the affected file, a description of the problem, and its impact on the user experience or system stability. The severity ratings follow a standard classification: Critical bugs cause crashes or data loss; High-priority issues cause significant UX degradation; Medium-priority issues are design concerns that should be addressed during recreation.

3.1 Critical Bugs (Must Fix)

Critical bugs represent failures that directly impact application stability, data integrity, or security. These issues would cause crashes, expose user data, or silently corrupt the application state if left unresolved. Each of these must be definitively addressed in the recreation, not merely patched.

#	File	Description	Impact
C1	Board.tsx	DragOverlay uses non-null assertion (cards.find()!) that crashes if card is deleted between drag start and overlay render	Runtime crash during DnD
C2	Sidebar.tsx	ChatPanel unmounts on tab switch, destroying SSE connection, message history, and session state	Chat data loss on navigation
C3	ChatPanel.tsx	Every streaming token re-renders entire message list, re-parsing markdown for all previous messages	Severe performance degradation
C4	ChatMessage.tsx	No rehype-sanitize on ReactMarkdown output from API responses creates XSS attack surface	Security vulnerability (XSS)
C5	.env.example	Contains real Supabase project URL and anon key instead of placeholder values	Credential exposure
C6	api.ts	response.json() has no try/catch; invalid JSON throws SyntaxError that bypasses error mapper	Unhandled crash on malformed API response

#	File	Description	Impact
C7	useChat.ts	No mutex on sendMessage; concurrent calls create competing AbortControllers with shared refs	Race condition in chat streaming

Table 5. Critical Bugs

C1 Fix - Safe DragOverlay Card Lookup: The current Board.tsx uses a dangerous non-null assertion on Array.prototype.find(), which returns undefined when no matching card is found. If the card is deleted between the drag start event and the DragOverlay render, the application crashes with a TypeError. The fix involves using optional chaining and providing a fallback.

```
// CURRENT (BUGGY): const activeCard = cards.find(c => c.id === activeId)!; {/*
CRASHES if activeCard is undefined */} // FIXED: const activeCard = cards.find(c =>
c.id === activeId); {activeCard ? : ( Card removed during drag )}
```

3.2 High-Priority Issues

High-priority issues cause significant UX degradation or represent architectural problems that will compound if not addressed. While they may not cause immediate crashes, they create fragile patterns that make the codebase difficult to maintain and extend. These should all be resolved during the recreation process through better architectural decisions.

#	File	Description	Impact
H1	ReviewPanel.tsx	No cancel/abort mechanism for hanging API calls; both Approve and Reject disabled during submit	User stuck on broken submit
H2	CardDrawer.tsx	Focus trap uses querySelector("button[onClick]") which is unreliable in React synthetic events	Broken focus management
H3	Card.tsx	handleSave and handleDelete do not call mutate() after success; card list does not refresh	Stale UI after actions
H4	Column.tsx	Badge shows active count but list still renders expired cards, confusing UX	Misleading count display
H5	Sidebar.tsx	No resize listener; isOpen initial state uses window.innerWidth once on mount	Broken responsive behavior
H6	CardDrawer.tsx	Drawer closes immediately after onDecision callback; user cannot see result	Confusing UX flow
H7	PublishConfirmModal.tsx	No double-click prevention on publish button after countdown reaches zero	Accidental double-publish
H8	vite.config.ts	No API proxy configuration; causes CORS issues during development	Dev environment broken
H9	MainLayout.tsx	No error boundary wrapping; any render error crashes entire application	Full app white-screen

#	File	Description	Impact
H10	api.ts	/api/kanban/cards/ vs /api/kanban/tasks/ - inconsistent resource naming	API contract confusion
H11	useAgentStream.ts	Unhandled loadHistory rejection; subscribe() never fires if history fetch fails	Missing real-time updates
H12	useChat.ts	No SSE reconnection for dropped connections; only handles timeouts	Permanent chat failure
H13	ScriptViewer.tsx	lg:grid-cols-2 never activates inside 600px-wide drawer (lg is 1024px)	Layout never triggers
H14	CompetitorCard.tsx	No loading="lazy" on thumbnails, no onError handler, no aspect-ratio	Broken image display

Table 6. High-Priority Issues

3.3 Medium-Priority Issues

Medium-priority issues represent design concerns, performance optimizations, and code quality improvements that should be addressed during recreation. While they do not cause immediate failures, they create technical debt that impacts maintainability, developer experience, and long-term application health.

#	File	Description
M1	Board.tsx	grouped cards computed every render without useMemo; recalculates on unrelated state changes
M2	Card.tsx	useCardTimer runs interval for every card even those not in column 2
M3	cardHelpers.ts	Column 1 has no action logic for error state; errored discovery cards get stuck
M4	constants.ts	EXPIRATION_MS hardcoded at 3 hours with no sync to backend 10-min cleanup cron
M5	usePipelineState.ts	Polls every 5s even when pipeline is idle or in terminal state
M6	useQuota.ts	Polls globally regardless of quota panel visibility
M7	useToast.ts	No maximum toast limit; rapid errors cause unlimited accumulation
M8	index.css	Duplicate animate-spin definition conflicts with Tailwind built-in
M9	types/index.ts	MODEL_REGISTRY hardcoded display data drifts from actual backend assignments
M10	errorMapper.ts	Missing 409 Conflict and 422 Unprocessable Entity handling
M11	supabase.ts	No runtime type validation on Supabase data; schema drift silently produces wrong data

#	File	Description
M12	useYouTube.ts	Dead code: apiFetcher function defined but never used
M13	KnowledgeBase.tsx	Same XSS concern as ChatMessage - renders Markdown without rehype-sanitize
M14	YouTubePanel.tsx	Competitor videos always fetched even when "Own Channel" tab is active
M15	multiple files	Retry counts, timeouts, backoff delays scattered across 4 files with no centralized config

Table 7. Medium-Priority Issues

4. Database Schema Reference

This section provides the complete database schema that the frontend must match. All Zod validation schemas, TypeScript type definitions, and API response types must align exactly with these table definitions. The schema is managed via Supabase migrations in the repository at supabase/migrations/001_initial_schema.sql. Any mismatch between frontend types and the database schema will cause silent data loss or runtime errors in the UI.

4.1 kanban_cards Table

This is the primary table driving the Kanban board UI. Each card represents a topic or content piece moving through the production pipeline. The column_index field uses integer values 1-6 to represent the six pipeline stages. The parent_id column creates a self-referencing foreign key that links discovery sub-task cards to their parent task card via the kanban_callback. The metadata field is a JSONB column that stores the topic_brief, viability_score, and other dynamic attributes. The frontend KanbanCard type MUST include parent_id (string | null) and must use column_index (NOT "column") as the field name to match the database exactly.

Column	Type	Constraints	Frontend Field
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	KanbanCard.id
title	TEXT	NOT NULL, DEFAULT empty string	KanbanCard.title
column_index	INTEGER	NOT NULL, DEFAULT 1, CHECK (1-6)	KanbanCard.column (MISMATCH!)
pipeline_run_id	TEXT	NULLABLE	Not in current types
parent_id	UUID	REFERENCES kanban_cards(id)	MISSING from current types!

Column	Type	Constraints	Frontend Field
color	TEXT	NOT NULL, DEFAULT "#1D9E75"	Not in current types
metadata	JSONB	DEFAULT '{}'::jsonb	KanbanCard.topic_brief
status	TEXT	NOT NULL, DEFAULT 'idle'	KanbanCard.status
position	INTEGER	NOT NULL, DEFAULT 0	Not in current types
expires_at	TIMESTAMPTZ	NULLABLE	KanbanCard.expires_at
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	KanbanCard.created_at
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT now(), auto-trigger	KanbanCard.updated_at

Table 8. kanban_cards Schema with Frontend Type Mapping

NOTE: CRITICAL SCHEMA MISMATCHES: (1) Frontend uses "column" but DB column is "column_index". (2) parent_id is missing from frontend types. (3) topic_brief and viability_score are stored inside the metadata JSONB column, NOT as top-level columns. The frontend must extract these from metadata after fetching.

Actual SQL from 001_initial_schema.sql:

```
CREATE TABLE IF NOT EXISTS kanban_cards ( id UUID PRIMARY KEY DEFAULT
gen_random_uuid(), title TEXT NOT NULL DEFAULT '', column_index INTEGER NOT NULL
DEFAULT 1 CHECK (column_index BETWEEN 1 AND 6), pipeline_run_id TEXT, parent_id UUID
REFERENCES kanban_cards(id), color TEXT NOT NULL DEFAULT '#1D9E75', metadata JSONB
DEFAULT '{}'::jsonb, status TEXT NOT NULL DEFAULT 'idle', position INTEGER NOT NULL
DEFAULT 0, expires_at TIMESTAMPTZ, created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now() );
```

4.2 agent_thoughts Table

This table stores the real-time activity log for each card. The frontend subscribes to INSERT events on this table via Supabase Realtime and appends new thoughts to an in-memory list. The thought_type column uses a CHECK constraint to enforce valid values: thinking, search, output, error, memory_read, and memory_write. The card_id foreign key has ON DELETE CASCADE, meaning thoughts are automatically deleted when their parent card is deleted. An index on card_id ensures fast queries when loading historical thoughts for a specific card.

Column	Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
card_id	UUID	REFERENCES kanban_cards(id) ON DELETE CASCADE

Column	Type	Constraints
agent_name	TEXT	NOT NULL, DEFAULT 'unknown'
thought_type	TEXT	NOT NULL, CHECK IN ('thinking','search','output','error','memory_read','memory_write')
content	TEXT	NOT NULL, DEFAULT empty string
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

Table 9. agent_thoughts Schema

4.3 pipeline_runs Table

This table tracks the execution state of LangGraph pipeline runs. The run_id is a text field that matches the LangGraph thread ID. The stage_outputs and stage_status fields use JSONB to store flexible per-stage data without requiring schema migrations when new stages are added. The frontend polls this table (via REST API) to show pipeline progress in the CardDrawer.

Column	Type	Constraints
run_id	TEXT	PRIMARY KEY
current_stage	TEXT	NOT NULL, DEFAULT 'trend_analysis'
stage_outputs	JSONB	DEFAULT '{}'::jsonb
stage_status	JSONB	DEFAULT '{}'::jsonb
status	TEXT	NOT NULL, DEFAULT 'running'
error_message	TEXT	DEFAULT ''

Table 10. pipeline_runs Schema

4.4 topic_briefs Table

This table serves as the Topic Reservoir where discovery-grade topics are stored after passing viability scoring. The viability_score_breakdown field stores the full 17-question scoring result as JSONB, which the frontend renders in the topic brief view inside the CardDrawer. The status field tracks whether a brief is in the reservoir, selected for production, or completed. This table is NOT directly rendered on the Kanban board but is referenced when viewing card details.

4.5 Supporting Tables

The schema also includes video_performance (tracks engagement metrics for published videos at 24h/7d/30d/90d intervals), production_cycles (manages the full lifecycle of a content piece from topic selection to publication, with lock_expires_at for concurrency control), escalations (tracks items requiring human intervention with a status CHECK constraint), and iteration_logs (records each scoring iteration in the Karpathy loop with scores, mutation zones, and script snapshots). All tables have Row Level Security enabled with permissive anon policies for service-side access. An update_updated_at_column() trigger automatically updates the timestamp on every row modification.

5. SSE Event Protocol Specification

The chat streaming system uses Server-Sent Events (SSE) over HTTP POST. The frontend sends a POST request to /api/chat/stream with a JSON body containing the user message and session_id. The backend responds with a streaming body where each line follows the SSE format: "data: {JSON}" followed by a newline. Each JSON payload has a "type" field that determines how the frontend should process the event. The frontend MUST handle all event types listed below; missing a handler for any type will cause silent data loss or broken UI states.

5.1 Chat Stream Events

These events are sent over the SSE connection from /api/chat/stream. The connection is initiated via POST (not GET) because the endpoint requires a JSON body. The frontend uses fetch() + ReadableStream (not EventSource) because EventSource only supports GET requests. The backend sends these events in the following order during a normal chat interaction.

Event Type	Direction	Payload Fields	Frontend Action
token	Server to Client	type: "token", content: string	Append content to streaming text buffer, update UI
tool_start	Server to Client	type: "tool_start", tool: string	Show "Calling {tool}..." indicator, add to active tools list
tool_end	Server to Client	type: "tool_end", tool: string	Remove from active tools list
done	Server to Client	type: "done", session_id?: string	Finalize message, persist session_id for conversation continuity
error	Server to Client	type: "error", message: string	Show error toast, append error text to message

Table 11. Chat SSE Event Types (Complete)

NOTE: There is NO "progress" event type. Pipeline progress is tracked through Supabase Realtime subscriptions on the kanban_cards and agent_thoughts tables, NOT through the SSE chat stream. Do not

implement a progress event handler.

SSE Client Implementation (from current useChat.ts): The current useChat.ts demonstrates the SSE parsing pattern. This pattern should be extracted into a reusable SSE client module in Chunk 1, with the addition of automatic reconnection logic.

```
// Actual SSE parsing from apps/web/src/hooks/useChat.ts const reader =
response.body.getReader(); const decoder = new TextDecoder(); let buffer = ''; let
accumulatedText = ''; while (true) { const { done, value } = await reader.read(); if
(done) break; buffer += decoder.decode(value, { stream: true }); const lines =
buffer.split('\n'); buffer = lines.pop() || ''; // Keep incomplete line in buffer for
(const line of lines) { if (!line.startsWith('data: ')) continue; const jsonStr =
line.slice(6).trim(); if (!jsonStr) continue; try { const event =
JSON.parse(jsonStr); switch (event.type) { case 'token': accumulatedText +=
event.content; setStreamingText(accumulatedText); break; case 'tool_start':
setActiveTools(prev => [...prev, event.tool]); break; case 'tool_end':
setActiveTools(prev => prev.filter(t => t !== event.tool)); break; case 'done': if
(event.session_id) sessionRef.current = event.session_id; setMessages(prev =>
prev.map(m => m.id === assistantId ? { ...m, content: accumulatedText } : m ));
break; case 'error': setError(event.message || 'An error occurred'); break; } } catch
{ /* Skip malformed JSON lines */ } } }
```

5.2 Agent Thought Events

Agent thoughts are received through a separate Supabase Realtime subscription, not through SSE. The frontend subscribes to INSERT events on the agent_thoughts table. When a new thought is inserted by the backend (during pipeline execution), the frontend receives the full row data and appends it to the in-memory thought list. The thought_type field determines the visual styling: "thinking" shows a blue indicator, "search" shows a green indicator, "output" shows a purple indicator, "error" shows a red indicator, "memory_read" and "memory_write" show yellow indicators.

6. Supabase Realtime Integration Patterns

Supabase Realtime provides WebSocket-based subscriptions to database changes. The frontend uses two subscriptions: one for kanban_cards (all CRUD operations) and one for agent_thoughts (INSERT only). Both subscriptions are managed through SWR to provide caching, revalidation, and automatic cleanup on component unmount. The patterns below show the actual implementation from the current codebase and highlight the improvements needed for the recreation.

6.1 Card Subscription Pattern

The useCards hook demonstrates the standard pattern for subscribing to Supabase Realtime changes. It creates a channel, registers a postgres_changes listener for all events on the kanban_cards table, and triggers an SWR mutate() call whenever any change occurs. This causes a full refetch of all cards, which is simple but potentially wasteful for large datasets. The recreation should add optimistic updates for drag-and-drop operations to reduce perceived latency.

```
// From apps/web/src/hooks/useCards.ts export function useCards() { const fetcher =
async (): Promise => { if (!supabase) return []; const { data, error } = await
supabase .from('kanban_cards') .select('*') .order('created_at', { ascending: false
}); if (error) throw error; return data as KanbanCard[]; }; const { data: cards,
error, isLoading, mutate } = useSWR('kanban-cards', fetcher); useEffect(() => { if
(!supabase) return; const channel = supabase .channel('kanban-realtime')
.on('postgres_changes', { event: '*', schema: 'public', table: 'kanban_cards' }, ()
=> { mutate(); } // Refetch all cards on any change ) .subscribe(); return () => {
supabase.removeChannel(channel); }; }, [mutate]); return { cards: cards ?? [],
isLoading, error, mutate }; }
```

6.2 Supabase Client Configuration

The Supabase client is initialized with the project URL and anon key from environment variables. The realtime configuration limits eventsPerSecond to 10 to prevent bandwidth overhead when the backend is generating many rapid updates. The client is exported as nullable (SupabaseClient | null) to allow the app to function in development mode without Supabase credentials. All hooks check isSupabaseConfigured() before attempting any operations.

```
// From apps/web/src/lib/supabase.ts import { createClient, SupabaseClient } from
'@supabase/supabase-js'; const supabaseUrl = import.meta.env.VITE_SUPABASE_URL; const
supabaseKey = import.meta.env.VITE_SUPABASE_ANON_KEY; export const supabase:
SupabaseClient | null = (supabaseUrl && supabaseKey) ? createClient(supabaseUrl,
supabaseKey, { realtime: { params: { eventsPerSecond: 10 } }, }) : null; export
function isSupabaseConfigured(): boolean { return supabase !== null; }
```

7. Backend LangGraph State Architecture

The backend uses two separate LangGraph StateGraphs, each with its own TypedDict state defined in packages/content_factory/orchestration/state.py. Understanding these states is essential for the frontend because the PipelineStateResponse type must match the ProductionState fields exactly. The backend snapshots state after every node transition, making the pipeline crash-proof: if the server dies at iteration 14 of 20, restart picks up exactly where it left off. The frontend polls this state to show pipeline progress in the CardDrawer.

7.1 DiscoveryState

The Discovery graph handles topic finding and viability grading. It has NO loops and NO human gates. The flow is: gather_context, search_web, generate_topics, grade_viability, save_topics. Once complete, it dumps topic cards into Kanban Column 2 (Suggested Topics) with a 3-hour expiration timer. The pipeline_status field transitions through "discovering", "grading", "complete", and "error" states.

Field	Type	Purpose
card_id	str	Identity - links to kanban_cards.id

Field	Type	Purpose
seed_hint	Optional[str]	Optional human hint like "AI in Pakistan"
zep_context	str	Past audience data from Zep memory
search_results	list	Raw Exa.ai search results
generated_topics	list	Topic dicts with titles/descriptions
graded_topics	list	Scored topics with viability results
pipeline_status	str	discovering | grading | complete | error
error	Optional[str]	Error message if pipeline failed

Table 12. DiscoveryState Field Map

7.2 ProductionState

The Production graph handles the full content creation pipeline with a Karpathy-inspired iterative refinement loop. It contains conditional loops (score threshold + iteration count), a human gate (interrupt for review/approval), and multi-stage progression. The pipeline_status field has a wider range of values reflecting the more complex flow: researching, drafting, scoring, mutating, visualizing, human_review, publishing, complete, and error. The frontend polls this state to show progress bars, current stage indicators, and iteration counts in the CardDrawer.

Field	Type	Purpose
card_id	str	Identity - links to kanban_cards.id
topic_brief	dict	Selected topic from Column 2
research_dossier	str	Deep research output
research_sources	list	Source URLs from research
zep_learnings	str	Cross-script learnings from Zep
current_draft	str	Current script draft being refined
best_draft	str	Highest-scoring draft across iterations
best_score	float	Score of best_draft (0-100)
evaluation_score	float	Score of current draft
evaluation_feedback	list	56-question binary checklist results
iteration_count	int	Current iteration in the refinement loop
visual_plan	str	Visual annotations (B-ROLL, MAP, etc.)

Field	Type	Purpose
human_feedback	Optional[str]	Human review feedback for revision
approved	bool	Whether human approved the script
revision_count	int	Number of revisions after human feedback
pipeline_status	str	Full status lifecycle (12 values)
error	Optional[str]	Error message if pipeline failed

Table 13. ProductionState Field Map

8. Chunk-by-Chunk Recreation Plan

The following plan divides the frontend recreation into 10 independently verifiable chunks. Each chunk is designed to produce a working, testable increment that can be validated against the existing backend before proceeding. The chunks are ordered by dependency: foundational layers (types, API client, layout) come first, followed by the core Kanban board, then detail views, and finally supplementary features. Each chunk specifies the files to create, the acceptance criteria for completion, the bugs it resolves, and the estimated scope.

Chunk	Name	Depends On	Est. Files	Priority
0	Project Foundation	None	12	Critical
1	Types and API Client	Chunk 0	8	Critical
2	Layout Shell	Chunk 0, 1	6	High
3	Kanban Board Core	Chunk 1, 2	10	Critical
4	Card Details and Drawer	Chunk 3	8	High
5	Review and Approval Flow	Chunk 4	5	Critical
6	Chat Panel	Chunk 1, 2	6	High
7	Sidebar Panels	Chunk 2, 6	12	Medium
8	Telemetry and System	Chunk 1, 2	6	Medium
9	Polish and Responsive	All chunks	Cross-cutting	Medium

Table 14. Chunk Dependency Map

8.1 Chunk 0: Project Foundation

Goal: Set up the project scaffolding, build configuration, and development infrastructure that all subsequent chunks depend on. This chunk establishes the Vite + React + TypeScript + Tailwind CSS v4 project with proper path aliases, API proxy, environment configuration, error boundaries, and a global toast system. No visible UI is produced in this chunk, but the foundation must be rock-solid before any components are built.

Key Decisions: Use React 18 (stable) instead of React 19 to avoid ecosystem compatibility issues. Add a Vite proxy for /api requests to eliminate CORS during development. Implement path aliases (@/ for src/) to improve import readability. Set up a centralized constants/config module that replaces the scattered hardcoded values across the current codebase. Add rehype-sanitize as a mandatory dependency for all markdown rendering.

CRITICAL DEPENDENCY SPECIFICATIONS (April 2026 audit):

```
// package.json - MUST use these exact versions "@supabase/supabase-js": "^2.101.0"
// Security fix (was ^2.45.0, 56 versions behind) "@dnd-kit/react": "^1.0.0" //
Replaces BOTH @dnd-kit/core AND @dnd-kit/sortable "tailwindcss": "^4.1.0" // v4.1.15
has container queries, 3D transforms "typescript": "^5.7.0" // Minor update "zod":
"^3.23.0" // Runtime type validation for API responses "rehype-sanitize": "^6.0.0" //
XSS prevention for markdown rendering "zustand": "^5.0.0" // Global UI state
management "date-fns": "^3.6.0" // Timestamp formatting "lucide-react": "^0.460.0" //
Icon library (replace emoji usage) "clsx": "^2.1.0" // Conditional classname utility
"@radix-ui/react-dialog": "^1.1.0" // Accessible drawer/modal primitives
"@radix-ui/react-tabs": "^1.1.0" // Accessible sidebar tab primitives
"react-error-boundary": "^4.0.0" // Error boundary with retry UI
"@tanstack/react-virtual": "^3.10.0" // Virtual scrolling for columns with 30+ cards
"swr": "^2.3.0" // Data fetching and caching // DO NOT install these deprecated
packages: // "@dnd-kit/core" (replaced by @dnd-kit/react) // "@dnd-kit/sortable"
(merged into @dnd-kit/react)
```

STATE MANAGEMENT (Zustand Store):

```
// src/lib/store.ts - Zustand for UI state only import { create } from 'zustand';
interface UIState { sidebarOpen: boolean; activeTab: number; // sidebar tab index
selectedCardId: string | null; drawerOpen: boolean; filters: { column: number | null;
search: string }; setSidebarOpen: (open: boolean) => void; setActiveTab: (tab:
number) => void; setSelectedCard: (id: string | null) => void; setDrawerOpen: (open:
boolean) => void; setFilters: (filters: Partial) => void; } export const useUIStore =
create((set) => ({ sidebarOpen: window.innerWidth >= 1024, activeTab: 0,
selectedCardId: null, drawerOpen: false, filters: { column: null, search: '' },
setSidebarOpen: (open) => set({ sidebarOpen: open }), setActiveTab: (tab) => set({
activeTab: tab }), setSelectedCard: (id) => set({ selectedCardId: id }),
setDrawerOpen: (open) => set({ drawerOpen: open }), setFilters: (f) => set((s) => ({
filters: { ...s.filters, ...f } })), }));
```

SWR remains the data-fetching layer (cards, pipeline state, quota). Zustand is only for UI state that does NOT come from API calls. Do NOT use React Context for this - Zustand's selector-based reactivity prevents unnecessary re-renders across components.

File	Purpose	Key Changes
package.json	Dependencies and scripts	Use exact versions listed above. Do NOT use old versions.
vite.config.ts	Vite build configuration	Add /api proxy to localhost:3000, path aliases (@/)
tsconfig.json	TypeScript configuration	ES2022 target, strict mode, path aliases
.env.example	Environment template	Replace real credentials with placeholders. Document API_AUTH_ENABLED=true.
.gitignore	Git ignore rules	Add .env to prevent credential commits
src/index.css	Global styles	Remove duplicate animate-spin, keep Tailwind v4 config
src/main.tsx	App entry point	Wrap in ErrorBoundary, use extensionless imports
src/App.tsx	Root component	Add ErrorBoundary wrapper at root level
src/lib/store.ts	Zustand store	NEW: UI state for sidebar, drawer, selected card, filters
src/lib/config.ts	Centralized constants	NEW: Consolidate all scattered constants
src/components/ErrorBoundary.tsx	Error boundary	Use react-error-boundary v4 with retry UI
src/hooks/useToast.ts	Toast store	Add max toast limit (5), use crypto.randomUUID()

Table 15. Chunk 0 Files

Acceptance Criteria:

- npm run dev starts without errors and serves the app at localhost:5173
- API proxy correctly forwards /api/* requests to localhost:3000 without CORS issues
- Path alias @/ resolves correctly (import { X } from "@/lib/config")
- .env.example contains only placeholder values, no real credentials
- ErrorBoundary catches render errors and shows a recovery UI instead of white-screen
- Toast system supports max 5 concurrent toasts with oldest-first dismissal
- TypeScript compiles in strict mode with zero errors (tsc --noEmit)

Bugs Resolved: C5 (credential exposure), C6 (API proxy + JSON handling), H8 (CORS proxy), H9 (error boundary)

8.2 Chunk 1: Types and API Client

Goal: Establish the complete type system and API client layer that all components will depend on. This chunk creates a comprehensive, Zod-validated type system that matches the backend API contracts exactly, and a robust HTTP client with proper error handling, retry logic, and timeout configuration. The API client

centralizes all endpoint definitions so that any contract mismatch is caught at compile time rather than at runtime.

Key Decisions: Use Zod for runtime type validation of API responses, not just TypeScript compile-time checks. This catches backend schema drift that TypeScript alone cannot detect. The API client should use an iterative retry approach instead of the current recursive pattern. The chat SSE client should be a separate module with its own reconnection logic.

Zod Schema for KanbanCard (matching DB schema exactly):

```
// src/types/index.ts import { z } from 'zod'; export const KanbanCardSchema =
z.object({ id: z.string().uuid(), title: z.string(), column_index:
z.number().int().min(1).max(6), // DB column name! pipeline_run_id:
z.string().nullable(), parent_id: z.string().nullable(), // CRITICAL: was missing
color: z.string().default('#1D9E75'), metadata: z.record(z.unknown()).default({}), //
topic_brief lives here status: z.string().default('idle'), position:
z.number().int().default(0), expires_at: z.string().datetime().nullable(),
created_at: z.string().datetime(), updated_at: z.string().datetime(), }); export type
KanbanCard = z.infer; // Helper to extract topic_brief from metadata export function
getTopicBrief(card: KanbanCard) { return (card.metadata as Record).topic_brief ??
null; } export function getViabilityScore(card: KanbanCard) { return (card.metadata
as Record).viability_score ?? null; }
```

File	Purpose	Key Changes
src/types/index.ts	Domain types + Zod schemas	Add Zod schemas, constrain column to 1-6, add parent_id
src/types/api.ts	API request/response schemas	NEW: Zod schemas for all 14 API endpoints
src/lib/api.ts	HTTP client with retry	Iterative retry, try/catch on JSON parse, configurable timeouts
src/lib/sse.ts	SSE streaming client	NEW: Reusable SSE client with reconnection, backoff
src/lib/supabase.ts	Supabase client	Keep existing null-safety pattern, add connection health check
src/lib/errorMapper.ts	Error mapping	Add 409/422 handling
src/lib/cardHelpers.ts	Card action logic	Add Column 1 error handling
src/lib/constants.ts	App constants	Import from centralized config, remove Groq references

Table 16. Chunk 1 Files

Bugs Resolved: C6 (JSON parse), C7 (SSE mutex via dedicated client), M3 (Column 1 action), M9 (MODEL_REGISTRY drift), M10 (409/422), M15 (scattered config)

8.3 Chunk 2: Layout Shell

Goal: Build the application layout skeleton that provides the visual structure for all subsequent chunks. This includes the Header with the topic discovery input, the MainLayout with proper flex arrangement, and the

Sidebar with tab navigation. The critical requirement is that the Sidebar must NOT unmount tab content when switching tabs (using CSS display:none instead of conditional rendering) to preserve chat state and SSE connections.

Key Decisions: Use CSS-based tab visibility (display:none/block) instead of React conditional rendering for sidebar tabs. This prevents the critical bug where ChatPanel loses its SSE connection on tab switch. Add a resize listener to the Sidebar that responds to window width changes.

Sidebar Tab Pattern (CSS-based visibility):

```
// src/layout/Sidebar.tsx - CRITICAL: CSS-based tabs preserve chat state const tabs =
[ { label: 'Chat', icon: MessageSquare, component: ChatPanel }, { label: 'YouTube',
icon: Youtube, component: YouTubePanel }, // ... more tabs ]; return (
{tabs.map((tab, i) => ( setActiveTab(i)}> {tab.label} ))} {/* CSS display:none
preserves component state! */} {tabs.map((tab, i) => ( ))} );
```

Bugs Resolved: C2 (chat state loss on tab switch), H5 (sidebar responsive)

8.4 Chunk 3: Kanban Board Core

Goal: Implement the central Kanban board with 6 columns, draggable cards, and real-time updates via Supabase Realtime. This is the most complex chunk and the visual centerpiece of the application. The board must support drag-and-drop between columns using the new @dnd-kit/react API, virtual scrolling for columns with many cards, and optimistic updates for smooth UX during drag operations.

Key Decisions: Migrate from @dnd-kit/core to @dnd-kit/react (^1.0.0). Add @tanstack/react-virtual for columns with 30+ cards. Use useMemo for expensive card grouping computations (fixes M1). Implement optimistic updates on drag end to prevent visual lag before Supabase Realtime confirms the move. Guard DragOverlay with optional chaining (fixes C1).

Bugs Resolved: C1 (DragOverlay crash), M1 (useMemo for grouped cards), H3 (stale UI after card actions)

8.5 Chunk 4: Card Details and Drawer

Goal: Implement the CardDrawer component that slides in when a card is clicked, showing detailed information about the card including its topic brief, agent activity log (from agent_thoughts), pipeline state progress, and script preview. The drawer uses Radix UI Dialog primitives for proper accessibility, focus trapping, and keyboard navigation (Escape to close).

Key Decisions: Use @radix-ui/react-dialog for the drawer foundation instead of custom focus trapping. This automatically handles focus trapping, Escape key dismissal, and screen reader announcements. The ScriptViewer should use responsive breakpoints appropriate for the drawer width (not lg:grid-cols-2 which requires 1024px in a 600px drawer). Add a 500ms delay before closing the drawer after approve/reject so the user can see the result.

Bugs Resolved: H2 (focus management), H6 (drawer closes immediately), H13 (ScriptViewer layout)

8.6 Chunk 5: Review and Approval Flow

Goal: Implement the review/approval workflow that allows users to review generated scripts, provide feedback, and approve or reject them for publication. This chunk integrates with the LangGraph human_review node via the resume endpoint, sending the human decision and optional feedback back to the backend to continue or terminate the production pipeline.

Key Decisions: Add an AbortController to the review submission so users can cancel if the API hangs (fixes H1). Disable the publish button after the first click to prevent double-publish (fixes H7). Show a brief confirmation animation before closing the drawer after decision.

Bugs Resolved: H1 (cancel hanging API), H7 (double-click publish)

8.7 Chunk 6: Chat Panel

Goal: Implement the streaming chat panel that connects to the /api/chat/stream SSE endpoint. The chat supports multi-turn conversations with session persistence, real-time token streaming, tool call indicators, and markdown rendering with XSS protection. This chunk directly fixes the most critical performance bug in the application.

Key Decisions: Memoize the ChatMessage component to prevent re-rendering all messages on every token (fixes C3). Add rehype-sanitize to all ReactMarkdown instances (fixes C4 and M13). Use the dedicated SSE client from Chunk 1 with automatic reconnection (fixes H12). Add a mutex using a ref-based lock to prevent concurrent sendMessage calls (fixes C7).

ChatMessage XSS Fix:

```
// src/components/chat/ChatMessage.tsx - XSS prevention import ReactMarkdown from
'react-markdown'; import remarkGfm from 'remark-gfm'; import rehypeSanitize from
'rehype-sanitize'; export function ChatMessage({ content, role }: ChatMessageProps) {
return ( ); }
```

Bugs Resolved: C3 (streaming re-render), C4 (XSS), C7 (chat mutex), H12 (SSE reconnection), M13 (KnowledgeBase XSS)

8.8 Chunk 7: Sidebar Panels

Goal: Implement the remaining sidebar tab panels: YouTube analytics (competitor videos and own channel stats), provider quota monitoring, model registry display, skill viewer, and knowledge base viewer. These are lower priority but complete the full dashboard experience.

Bugs Resolved: H14 (CompetitorCard image handling), M12 (dead code), M14 (YouTube fetch optimization)

8.9 Chunk 8: Telemetry and System

Goal: Implement performance telemetry, polling optimizations, and system-level utilities. This chunk addresses the medium-priority performance issues by adding smart polling (pause when idle), virtual scrolling metrics, and centralized configuration for all retry/timeout values.

Bugs Resolved: M2 (timer for non-expiring cards), M4 (expiration sync), M5 (idle polling), M6 (panel-aware polling), M7 (toast limit), M8 (duplicate CSS), M15 (centralized config)

8.10 Chunk 9: Polish and Responsive

Goal: Final polish pass across all components. This includes responsive breakpoints for mobile and tablet layouts, keyboard navigation improvements, ARIA labels, reduced-motion preferences, loading skeleton states, empty state illustrations, and cross-browser testing. All previous bugs should be verified as resolved in this chunk.

Acceptance Criteria: All 7 critical bugs, 14 high-priority issues, and 15 medium-priority improvements verified as resolved. Application functions correctly at 320px, 768px, 1024px, and 1440px viewport widths. Lighthouse accessibility score >= 90.

9. API Contract Reference

The following table lists all API endpoints consumed by the frontend. Each endpoint is documented with its HTTP method, path, request/response types, and authentication requirements. The frontend api.ts module must always send the X-API-Key header because API_AUTH_ENABLED defaults to TRUE in the backend (packages/core/config.py line 101) when API_KEYS is non-empty.

Method	Endpoint	Purpose	Auth
POST	/api/discover	Start topic discovery pipeline	X-API-Key
POST	/api/produce	Start production pipeline for a card	X-API-Key
POST	/api/resume	Resume pipeline after human gate	X-API-Key
GET	/api/pipeline/{id}	Get pipeline state for a card	X-API-Key
GET	/api/quota	Get provider quota status	X-API-Key
GET	/api/skills	Get available pipeline skills	X-API-Key
PUT	/api/kanban/cards/{id}	Update a card (save topic, edit title)	None
DELETE	/api/kanban/cards/{id}	Delete a card	None
PATCH	/api/kanban/cards/{id}/move	Move card to different column	None
GET	/api/youtube/competitors	Get competitor video analytics	X-API-Key
GET	/api/youtube/own	Get own channel analytics	X-API-Key
POST	/api/chat/stream	SSE chat streaming endpoint	X-API-Key
POST	/api/youtube/repurpose	Repurpose a video for new topic	X-API-Key
GET	/api/knowledge-base	Get knowledge base entries	X-API-Key

Table 17. API Contract Reference

NOTE: AUTHENTICATION: API_AUTH_ENABLED defaults to TRUE (packages/core/config.py line 101). Auth is enforced when API_AUTH_ENABLED=true AND API_KEYS is non-empty. The frontend api.ts must ALWAYS send X-API-Key header - never assume auth is disabled.

10. Error Boundary Strategy

The application implements a 3-tier error boundary strategy to prevent cascading failures and provide graceful degradation at every level of the component tree. Each tier catches different types of errors and provides appropriate recovery actions.

Tier	Scope	Catches	Recovery Action
Tier 1	Root (App.tsx)	Unrecoverable render errors, failed lazy loads	Full-page error with "Reload" button
Tier 2	Feature sections (Board, Sidebar, Drawer)	Component-level render errors	Section-level fallback with retry button
Tier 3	Individual widgets (ChatMessage, Card, AgentLog)	Data rendering errors (bad markdown, missing fields)	Inline error indicator, component continues

Table 18. Error Boundary Tiers

Tier 1 - Root Error Boundary Implementation:

```
// src/components/ErrorBoundary.tsx import { ErrorBoundary } from
'react-error-boundary'; function ErrorFallback({ error, resetErrorBoundary }:
ErrorFallbackProps) { return ( Something went wrong {error.message} {
console.error(error); resetErrorBoundary(); }} className="px-4 py-2 bg-blue-600
text-white rounded-lg hover:bg-blue-700" >Reload Application ); } // In main.tsx:
window.location.reload()} >
```

For async errors (Supabase disconnect, SSE failure, network timeout), the application registers global error handlers in main.tsx that log to console.error and display toast notifications via the useToast hook. These handlers catch errors that bypass React's error boundary system.

11. Testing Strategy

The testing strategy follows a layered approach that balances speed, coverage, and confidence. Unit tests validate individual hooks and utility functions in isolation using mocked dependencies. Integration tests

verify that components interact correctly with SWR, Supabase, and the API client. Visual regression tests catch unintended layout changes using screenshot comparison. End-to-end tests validate critical user flows (discover topic, review script, publish) against the actual backend.

Layer	Tool	Scope	Target Coverage
Unit	Vitest	Hooks (useCards, useChat, usePipelineState), utilities (cardHelpers, errorMapper, config)	80% of non-component code
Component	Vitest + Testing Library	All 25+ components render without crashing, handle edge cases	Critical path components
Integration	MSW (Mock Service Worker)	SWR cache behavior, Supabase Realtime subscriptions, SSE streaming	All data-fetching hooks
Visual	Playwright screenshots	Board layout, sidebar tabs, drawer, responsive breakpoints	3 viewports (768, 1024, 1440)
E2E	Playwright	Discover topic -> Review script -> Approve -> Publish flow	5 critical user flows

Table 19. Testing Strategy Layers

12. Risk Mitigation

The frontend recreation carries several risks that must be actively managed throughout the process. The following table identifies the top risks, their probability and impact, and the specific mitigation strategies that should be employed to reduce their likelihood or impact.

Risk	Probability	Impact	Mitigation
@dnd-kit/react API instability	Medium	High	Pin to exact version, write abstraction layer to isolate DnD code from components
Supabase schema changes during rebuild	Medium	High	Use Zod schemas as contract tests, add CI check that validates schemas against DB
Backend API contract drift	Low	High	Centralize all endpoint definitions in api.ts with strict types, add API contract tests
React 19 ecosystem incompatibility	Medium	Medium	Consider React 18 if critical packages lack React 19 support; test early in Chunk 0
Tailwind CSS v4 breaking changes	Low	Low	CSS-first approach is stable, but pin tailwindcss version and test utility classes early

Risk	Probability	Impact	Mitigation
SSE connection instability	Medium	Medium	Implement exponential backoff reconnection with max 5 retries in dedicated SSE client
Scope creep from feature requests	High	Medium	Strict chunk gates: each chunk must pass acceptance criteria before proceeding

Table 20. Risk Assessment and Mitigation

13. Deployment and Environment Setup

This section provides the complete environment setup guide for both development and production deployments. The frontend is a Vite-built React SPA that communicates with the FastAPI backend and Supabase directly from the browser. The deployment architecture uses a single Vite build output served as static files, with the backend API proxying through a reverse proxy (Nginx or Cloudflare) in production.

13.1 Development Environment

The development environment requires Node.js 20+, npm 10+, and access to the Supabase project. The Vite dev server runs at localhost:5173 and proxies all /api/* requests to the FastAPI backend at localhost:3000. The following steps set up the complete development environment.

```
# 1. Clone and switch to the correct branch git clone
https://github.com/HassanArif-collab/AI-Orchestration.git cd AI-Orchestration git
checkout codebase-audit-finding-fixes # 2. Install frontend dependencies cd apps/web
npm install # 3. Set up environment variables cp .env.example .env # Edit .env with
your Supabase credentials and API key # 4. Start backend (in separate terminal) cd
apps/api pip install -r requirements.txt uvicorn main:app --reload --port 3000 # 5.
Start frontend dev server cd apps/web npm run dev # http://localhost:5173
```

13.2 Environment Variables

The following environment variables are required for the frontend. All sensitive values must be stored in .env (gitignored) and referenced via VITE_ prefixed variables in Vite. The .env.example file must contain only placeholder values - NEVER commit real credentials.

Variable	Required	Description
VITE_SUPABASE_URL	Yes	Supabase project URL (e.g., https://xxx.supabase.co)
VITE_SUPABASE_ANON_KEY	Yes	Supabase anonymous key for client-side access
VITE_API_URL	No	Backend API base URL (defaults to /api proxy in dev)

Variable	Required	Description
VITE_API_KEY	Yes	API authentication key (X-API-Key header)

Table 21. Environment Variables

13.3 Production Build

The production build uses Vite to create an optimized static bundle. The build output is served directly by a web server (Nginx, Vercel, or Netlify). API requests are handled by a reverse proxy configuration that forwards /api/* to the FastAPI backend. The Supabase client connects directly from the browser to the Supabase cloud service, bypassing the backend entirely for Realtime subscriptions and direct database queries.

```
# Production build cd apps/web npm run build # Output in dist/ # Preview production build locally npm run preview # http://localhost:4173 # Nginx configuration (production) server { listen 80; root /var/www/frontend/dist; location /api/ { proxy_pass http://localhost:3000; } location / { try_files $uri $uri/ /index.html; } }
```